```
> VerifyCriticalPoint := proc()
    local DerivativeInterval, midpoint, radius, q0, w0, eps, Q,
W,
          Q0, W0, f0, J0, JX, a, b, c, d, determinant, Y, i, j,
          diagonal, rowBound, imageBound, rowUpper, imageUpper,
          rowLimit, imageLimit;

    # Enclose partial_w^iw partial_q^iq Psi on a positive
rectangle.
    DerivativeInterval := proc(qRange, wRange, iw, iq)
        local k, e, coefficient, term1, term2, lower, upper,
              firstOmitted, firstExponent, tail;
        lower, upper := 0, 0;
        for k from 0 to 12 do
            e := k*(k-1)/2;
            if k >= iw and e >= iq then
                coefficient := (-1)^k*binomial(k,
iw)*factorial(iw)
                                          *binomial(e,
iq)*factorial(iq);
                term1 :=
coefficient*wRange[1]^(k-iw)*qRange[1]^(e-iq);
                term2 :=
coefficient*wRange[2]^(k-iw)*qRange[2]^(e-iq);
                lower := lower + min(term1, term2);
                upper := upper + max(term1, term2);
            end if;
        end do;

        # For 1/4 < q < 1/3 and 1 < w < 3, bound each omitted
        # term by k^iw*e^iq*4^iq*3^(k-e).
        firstOmitted := 13;
        firstExponent := firstOmitted*(firstOmitted-1)/2;
        if 2^(iw+2*iq+1) >= 3^(firstOmitted-1) then
            error "the geometric tail ratio is not below 1/2";
        end if;
        tail := 2*firstOmitted^iw*firstExponent^iq*4^iq
                 / 3^(firstExponent-firstOmitted);
        return [lower-tail, upper+tail];
    end proc;

    midpoint := B -> (B[1]+B[2])/2;
    radius := B -> (B[2]-B[1])/2;

    q0 := 3092493386004/10^13;
    w0 := 2320376944290/10^12;
```

```
    eps := 1/10^12;
    Q := [q0-eps, q0+eps];
    W := [w0-eps, w0+eps];
    if Q[1] <= 309/1000 or Q[2] >= 1237/4000 or
       W[1] <= 2 or W[2] >= 233/100 then
        error "the box lies outside the stated bounds";
    end if;

    Q0, W0 := [q0, q0], [w0, w0];
    f0 := [DerivativeInterval(Q0, W0, 0, 0),
           DerivativeInterval(Q0, W0, 1, 0)];
    J0 := [[DerivativeInterval(Q0, W0, 0, 1), f0[2]],
           [DerivativeInterval(Q0, W0, 1, 1),
            DerivativeInterval(Q0, W0, 2, 0)]];
    JX := [[DerivativeInterval(Q, W, 0, 1),
DerivativeInterval(Q, W, 1, 0)],
           [DerivativeInterval(Q, W, 1, 1),
DerivativeInterval(Q, W, 2, 0)]];

    a, b := midpoint(J0[1][1]), midpoint(J0[1][2]);
    c, d := midpoint(J0[2][1]), midpoint(J0[2][2]);
    determinant := a*d-b*c;
    if determinant = 0 then
        error "the midpoint matrix is singular";
    end if;
    Y := [[d/determinant, -b/determinant],
          [-c/determinant, a/determinant]];

    rowLimit := [234/10^13, 301/10^12];
    imageLimit := [323/10^15, 397/10^15];
    printf("Coordinate order (q,w)\n");
    for i from 1 to 2 do
        rowBound := 0;
        for j from 1 to 2 do
            diagonal := 0;
            if i = j then
                diagonal := 1;
            end if;
            rowBound := rowBound
                + abs(diagonal - Y[i][1]*midpoint(JX[1][j])
                               - Y[i][2]*midpoint(JX[2][j]))
                + abs(Y[i][1])*radius(JX[1][j])
                + abs(Y[i][2])*radius(JX[2][j]);
        end do;
        imageBound := eps*rowBound
            +
```

```
  abs(Y[i][1]*midpoint(f0[1])+Y[i][2]*midpoint(f0[2]))
              + abs(Y[i][1])*radius(f0[1]) +
  abs(Y[i][2])*radius(f0[2]);

          # Round the displayed upper bounds upward with exact
  arithmetic.
          rowUpper := ceil(10^18*rowBound);
          imageUpper := ceil(10^18*imageBound);
          if rowUpper/10^18 >= rowLimit[i] or
             imageUpper/10^18 >= imageLimit[i] then
              error "a printed bound fails for coordinate %1", i;
          end if;
          printf("Row %d sum <= 0.%018d   PASS\n", i, rowUpper);
          printf("Coordinate %d image distance <= 0.%018d
  PASS\n",
                  i, imageUpper);
      end do;

      printf("Box inclusion and contraction verified.\n");
      printf("|rho - 0.3092493386004| < 10^(-12)\n");
      printf("|tau - 2.320376944290| < 10^(-12)\n");
  end proc:

  VerifyCriticalPoint():
Coordinate order (q,w)
Row 1 sum <= 0.000000000023328356   PASS
Coordinate 1 image distance <= 0.000000000000322520   PASS
Row 2 sum <= 0.000000000300089405   PASS
Coordinate 2 image distance <= 0.000000000000396910   PASS
Box inclusion and contraction verified.
|rho - 0.3092493386004| < 10^(-12)
|tau - 2.320376944290| < 10^(-12)
>
>
```